

16주차 스터디

교재 : 딥러닝 파이토치 교과서

분량 : 8장

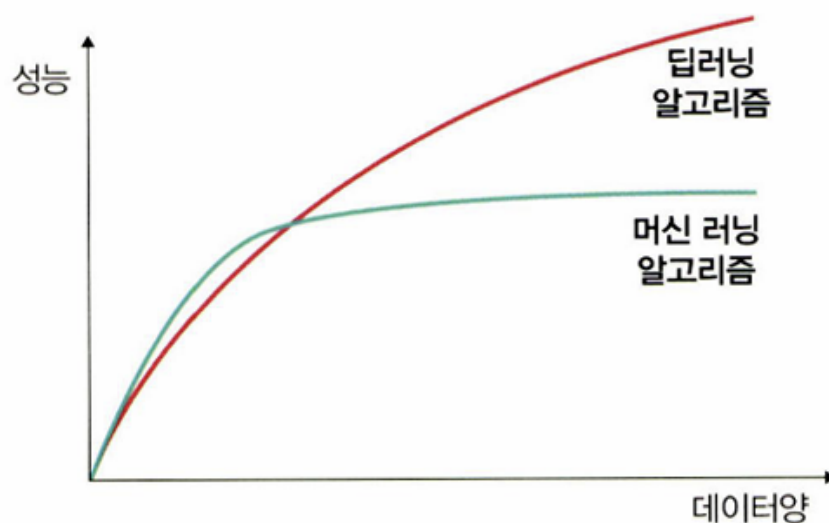
과제 기간 : ~2025-06-22

8.1 성능 최적화

8.1.1 데이터를 사용한 성능 최적화

- 최대한 많은 데이터 수집하기
 - 일반적으로 DL, ML은 데이터양이 많을수록 성능이 좋다

▼ 그림 8-1 데이터와 딥러닝, 머신 러닝 알고리즘의 성능 비교



- 데이터 생성하기 : 많은 데이터를 수집할 수 없다면 데이터를 만들어 사용한다
- 데이터 범위(scale) 조정하기
 - 활성화 함수로 시그모이드를 사용한다면 데이터셋 범위 0~1
 - 하이퍼볼릭 탄젠트를 사용한다면 데이터셋 범위 -1~1을 갖도록 조정
 - 정규화, 규제화, 표준화도 성능 향상에 도움이 된다

8.1.2 알고리즘을 이용한 성능 최적화

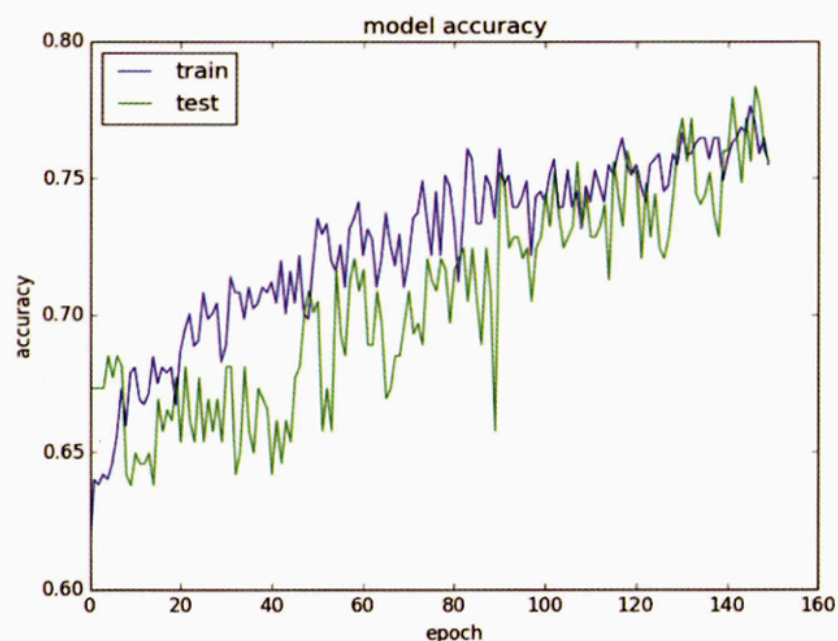
- 유사한 용도의 알고리즘들을 선택해 모델을 훈련시켜 본 뒤, 최적의 성능인 알고리즘을 선택해야 한다
- ML - 데이터 분류 : SVM, K-Means 등의 알고리즘을 훈련시켜 선택
- 시계열 데이터 - RNN, LSTM, GRU 등의 알고리즘을 훈련시켜 선택

8.1.3 알고리즘 튜닝을 위한 성능 최적화

- 성능 최적화를 하는 데 가장 많이 시간이 소요되는 부분
- 모델 하나 선택해서 하이퍼파라미터를 변경하면서 훈련시키고 최적의 성능을 도출해야 한다
- 선택할 수 있는 하이퍼파라미터 항목
 - **진단** : 성능 향상이 어느 순간 멈췄다면 원인을 분석해야 한다. 문제를 진단하는데 사용할 수 있는 것이 모델에 대한 평가이다

평가 결과를 바탕으로 모델이 과적합인지 다른 이유로 성능 향상에 문제가 있는지 에 대한 인사이트를 얻는다

▼ 그림 8-2 알고리즘 성능 진단



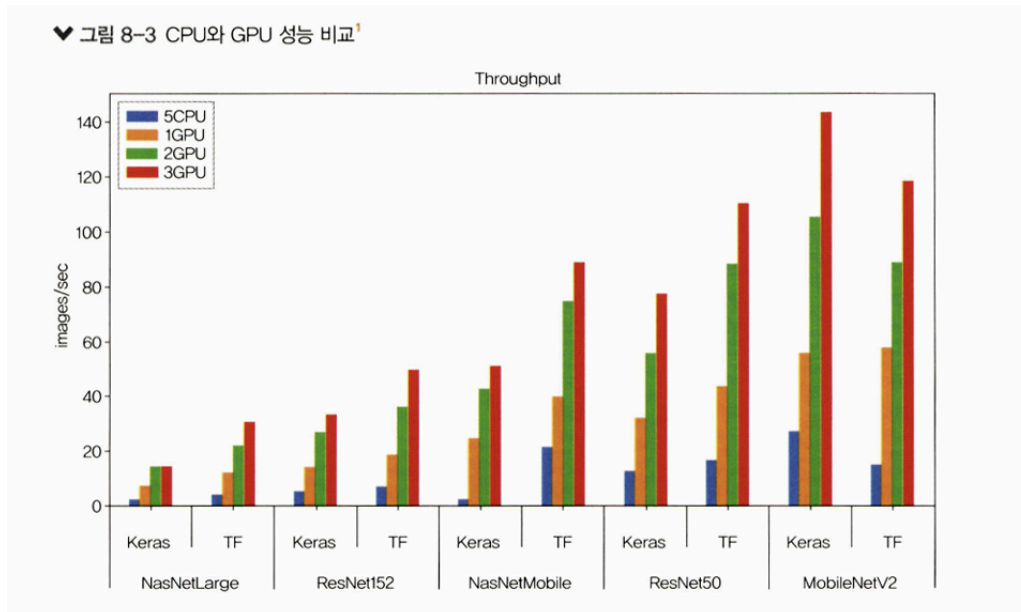
- 훈련 성능이 검증(test)보다 눈에 띄게 좋다면 과적합 의심
 - 규제화 진행, 성능 향상에 도움이 됨
 - 훈련과 검증 모두 성능이 좋지 않다면, 과소적합 의심
 - 네트워크 구조 변경, 에포크 수 조정해서 훈련 늘리기
 - 훈련 성능이 검증을 넘어서는 변곡점이 있으면 조기 종료를 고려하기
- **가중치** : 가중치에 대한 초기값은 작은 난수를 사용한다. 오토인코더 같은 비지도 학습을 이용해 사전 훈련을 진행한 후 지도 학습을 진행하기도 한다
 - **학습률** : 모델의 네트워크 구성에 따라 다르기 때문에 초기에 매우 크거나 작은 임의의 난수를 선택해 결과를 보고 조금씩 변경해줘야 한다.
 네트워크 계층이 많다면 학습률이 높아야하고 계층이 몇 개 안되면 작게 설정해야 한다.
 - **활성화 함수** : 신중히 변경해야 한다(손실 함수도 함께 변경해야 하는 경우가 많아서). 따라서 다루고자 하는 데이터의 유형과 얻고 싶은 결과를 명확히 이해하고 함수를 변경하자.
 일반적으로는 활성화 함수는 시그모이드 / 하이퍼볼릭 탄젠트, 출력층에서는 소프트맥스 / 시그모이드 함수 사용
 - **배치와 에포크** : 적절한 배치 크기를 위해 훈련 데이터셋의 크기와 동일하게 하거나 하나의 배치로 훈련시켜 보는 등 다양한 테스트를 진행해라
 - **옵티마이저 및 손실 함수** : 일반적으로 옵티마이저는 확률적 경사 하강법 많이 사용. Adam, RMSprop도 좋은 성능을 보임
 다양한 옵티마이저와 손실 함수를 적용해보고 성능이 최고인 것을 선택한다
 - **네트워크 구성(네트워크 토폴로지)** : 네트워크의 구성을 변경해 가면서 성능을 테스트한다
 ex) 하나의 은닉층에 뉴런 여러 개를 포함시키거나, 네트워크 계층은 늘리고 뉴런 개수는 줄여보기 등

8.1.4 앙상블을 이용한 성능 최적화

- 앙상블 : 모델을 두 개 이상 섞어서 사용하는 것
- 앙상블을 이용하는 것도 성능 향상에 도움이 됨

8.2 하드웨어를 이용한 성능 최적화

8.2.1 CPU와 GPU 사용의 차이



- CPU :

- 연산 담당 ALU와 명령어를 해석하고 실행하는 control, 데이터를 담아 두는 cache로 구성
- 명령어가 입력되는 순서대로 데이터를 처리하는 직렬 처리 방식
- 한 번에 하나의 명령어만 처리 → 연산을 담당하는 ALU의 개수가 많을 필요 X

- GPU :

- 병렬 처리를 위해 개발 → 캐시 메모리 비중은 낮고, 연산을 수행하는 ALU의 개수는 증가
- 서로 다른 명령어를 동시에 병렬적으로 처리하도록 설계, 여러 명령을 동시에 처리하는 병렬 처리 방식에 특화
- 하나의 코어에 ALU 수백~수천 개가 장착
- CPU로는 시간이 많이 걸리는 3D 그래픽 작업 등을 빠르게 수행 가능

- 개별적 코어 속도는 CPU >>>> GPU

- CPU가 적합한 분야와 GPU가 적합한 분야가 나뉘어진다

- ex) A열 > B열 > C열 순서대로 처리되는 순차적 연산 : CPU 적합

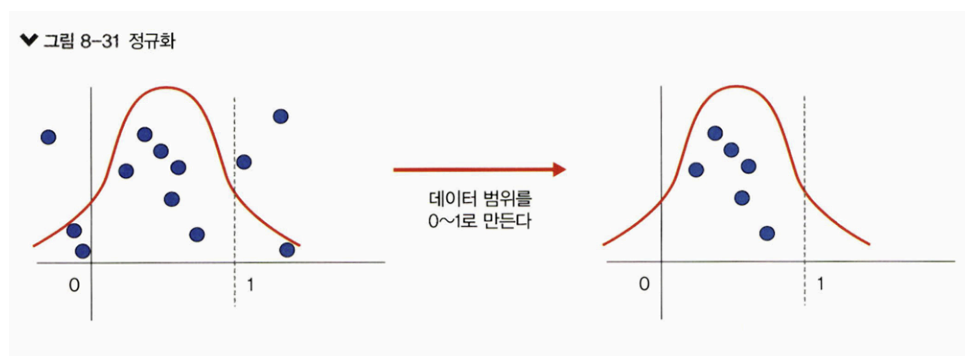
- 역전파처럼 복잡한 미적분의 병렬 연산 : GPU 적합

8.3 하이퍼파라미터를 이용한 성능 최적화

8.3.1 배치 정규화를 이용한 성능 최적화

<정규화>

- normalization, 데이터 범위를 사용자가 원하는 범위로 제한하는 것
- 예) 이미지 데이터 픽셀 정보 0~255를 0~1.0 사이의 값으로 변환



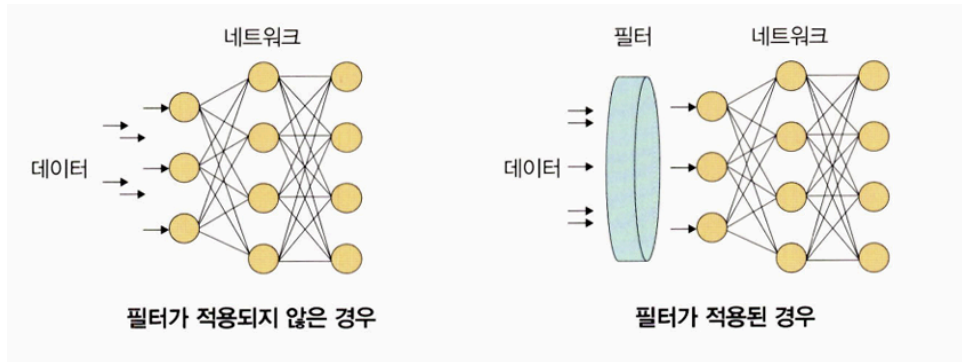
- 각 특성 범위(스케일, scale)를 조정한다는 의미로, 특성 스케일링(feature scaling)이라고도 함
- 스케일 조정을 위해 MinMaxScaler() 기법을 사용한다

$$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

(x: 입력 데이터)

<규제화>

- regularization, 모델 복잡도를 줄이기 위해 제약을 두는 방법
- 제약 : 데이터가 네트워크에 들어가기 전 필터를 적용한 것
- 예) 필터로 걸러진 데이터만 네트워크에 투입해 빠르고 정확하게 결과를 얻기

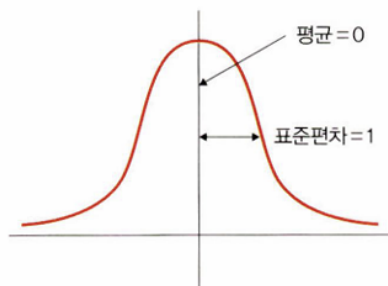


- 규제를 이용해 모델 복잡도를 줄이는 방법은 **1. 드롭아웃** **2. 조기종료** 이다.

<표준화>

- standardization, 기존 데이터를 평균은 0, 표준 편차는 1인 형태의 데이터로 만드는 방법
- 표준화 스칼라(standard scaler) 혹은 z-스코어 정규화(z-score normalization) 이라고도 한다

▼ 그림 8-33 표준화



- 평균을 기준으로 얼마나 떨어져 있는지 살펴볼 때 사용
- 보통 데이터 분포가 가우시안 분포를 따를 때 유용한 방법으로 다음 수식을 사용한다

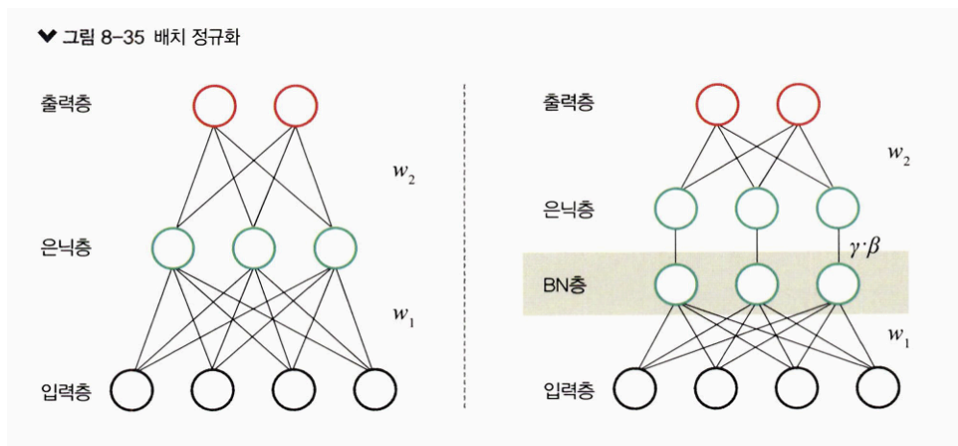
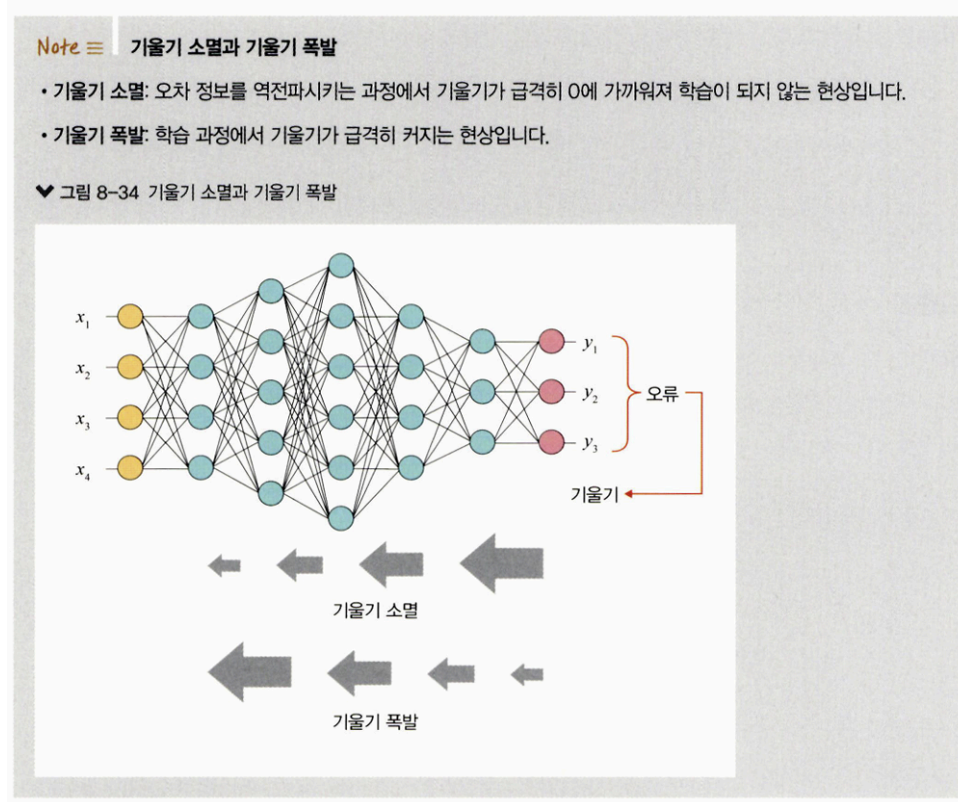
$$\frac{x - m}{\sigma}$$

(σ : 표준편차, x : 관측 값(입력 값), m : 평균)

<배치 정규화>

- **batch normalization**, 데이터 분포가 안정되어 학습 속도를 높일 수 있다

- 기울기 소멸/폭발 문제를 해결하기 위한 방법
- 일반적으로 기울기 소멸/폭발 문제를 해결하기 위해 손실 함수로 ReLU를 사용하거나 초깃값 튜닝, 학습률 등을 조정한다



- 배치 정규화가 소개된 논문에 따르면 기울기 소멸/폭발의 원인은 내부 공변량 변화 (internal covariance shift) 때문
 - 네트워크의 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀌는 현상

- 따라서 분산된 분포를 정규 분포로 만들기 위해 표준화와 유사한 방식을 **mini-batch**에 적용
- **평균은 0, 표준 편차는 1**로 유지하도록 수식을 적용한다

$$\mu\beta \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \text{----①}$$

$$\sigma^2\beta \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 \quad \text{----②}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} \quad \text{----③}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) \quad \text{----④}$$

$$\left(\begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)$$

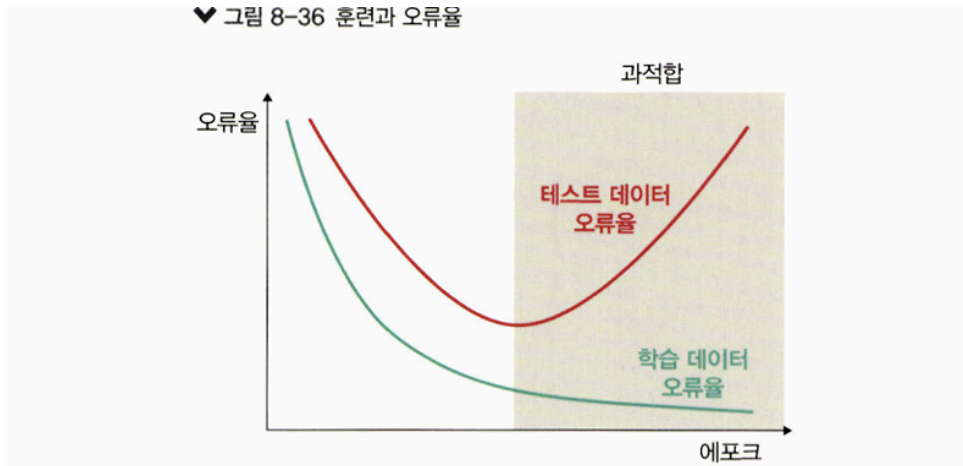
- ① 미니 배치 평균을 구합니다.
- ② 미니 배치의 분산과 표준편차를 구합니다.
- ③ 정규화를 수행합니다.
- ④ 스케일(scale)을 조정(데이터 분포 조정)합니다.

- 매 단계마다 활성화 함수를 거치며 **데이터셋 분포가 일정해진다** → **속도 향상** 가능
- 단점 :
 1. 배치 크기가 작을 때는 정규화 값이 기존 값과 다른 방향으로 훈련될 수 있다
예) 분산이 0이면 정규화 자체가 안되는 경우 발생
 2. RNN은 네트워크 계층별로 미니 정규화를 적용해야 하기 때문에 모델이 더 복잡해질 수 있다

8.3.2 드롭아웃을 이용한 성능 최적화

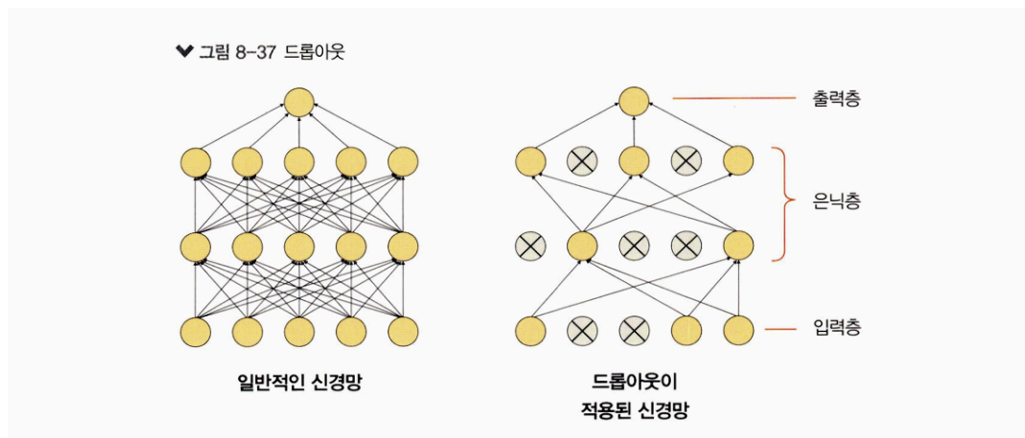
- 과적합 : 훈련 데이터셋을 과하게 학습하는 것
- 문제인 이유 : 훈련 데이터셋에 대해 훈련을 계속하면 → 오류는 줄어들지만, 테스트 데이터셋에 대한 오류는 어느 순간 증가한다

♥ 그림 8-36 훈련과 오류율



- **드롭아웃(dropout)** : 훈련할 때 일정 비율의 뉴런만 사용하고, 나머지 뉴런에 해당하는 가중치는 업데이트 하지 않는 방법
 - 매 단계마다 사용하지 않는 뉴런은 바꾸며 훈련시킴
 - 드롭아웃은 노드를 임의로 끄면서 학습하는 방법
 - 은닉층에 배치된 노드 중 일부를 임의로 끄면서 학습
 - 꺼진 노드는 신호를 전달하지 않으므로 지나친 학습을 방지하는 효과가 생김

♥ 그림 8-37 드롭아웃

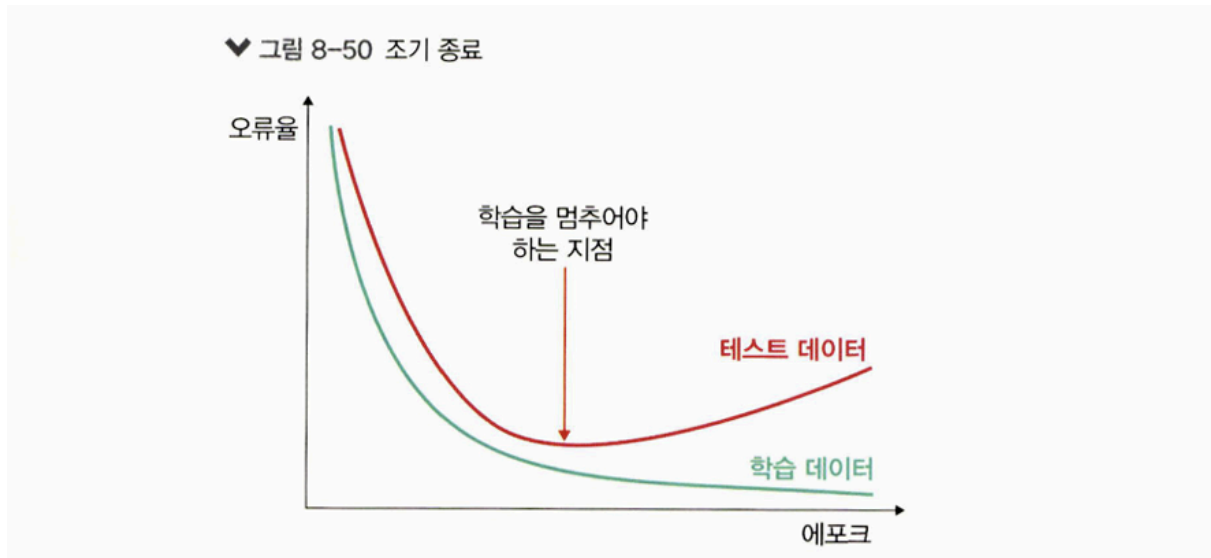


8.3.3 조기 종료를 이용한 성능 최적화

- **조기 종료(early stopping)** : 뉴럴 네트워크가 과적합을 회피하는 규제 기법
- 훈련 데이터와 별도로 검증 데이터를 준비
- 매 에포크마다 검증 데이터에 대한 오차(validation loss)를 측정하여 모델의 종료 시점을 제어

⇒ 즉, 과적합이 발생하기 전까지 학습에 대한 오차(training loss)와 검증에 대한 오차 모두 감소하지만, 과적합이 발생하면 훈련 데이터셋에 대한 오차는 감소하는 반면 검증 데이터셋에 대한 오차는 증가합니다

⇒ 따라서 조기 종료는 **검증 데이터셋에 대한 오차가 증가하는 시점에** 학습을 멈추도록 조정한다



- 조기 종료는 학습을 언제 종료시킬지만 결정한다
- 최고의 성능의 모델을 보장하는 방법은 아니라는 것에 주의해야 한다.